```
> ##############################################################
  #  monopole_wave_maple.mpl
  #  Magnetic Monopole Wave Equation — TauTimp / Teoria Onduscularа
  #  R = 3.9228e22 m   (electron-scale boundary parameter)
  #  Author : Viorel Popescu
  #  Execute : !!! (Execute All) in Maple 2022+
  ##############################################################

  restart;
  with(plots): with(plottools): with(LinearAlgebra):

  ##############################################################
  # SECTION 0 — Physical constants and key numerics
  ##############################################################
  printf("=== SECTION 0 : Physical constants ===\n");

  c    := 2.99792458e8:          # speed of light  [m/s]
  mu0  := 4*Pi*1e-7:             # vacuum permeability  [N/A^2]
  hbar := 1.054571817e-34:       # reduced Planck constant  [J•s]
  e_ch := 1.602176634e-19:       # elementary charge  [C]
  R    := 3.9228e22:             # boundary parameter  [m]

  # Zeros of J_0  (Bessel first kind, order 0)
  j01 := 2.404825558:
  j02 := 5.520078110:
  j03 := 8.653727913:

  # Wave-numbers  k_n = j_{0,n} / (2*sqrt(2))
  k1 := j01 / (2*sqrt(2)):
  k2 := j02 / (2*sqrt(2)):
  k3 := j03 / (2*sqrt(2)):

  # Angular frequencies
  omega1 := evalf(c * k1):
  omega2 := evalf(c * k2):
  omega3 := evalf(c * k3):

  # Dirac magnetic charge
  g_D := evalf(2*Pi*hbar / (mu0 * e_ch)):

  printf("  k1      = %g  m^-1\n", k1);
  printf("  k2      = %g  m^-1\n", k2);
  printf("  k3      = %g  m^-1\n", k3);
  printf("  omega1  = %g  rad/s\n", omega1);
  printf("  omega1  = %g  rad/s\n", omega2);
  printf("  omega3  = %g  rad/s\n", omega3);
  printf("  g_D     = %g  A*m\n",   g_D);
  printf("  freq ratio omega2/omega1 = %g  (theory: j02/j01 = %g)\n",
```

```
       evalf(omega2/omega1), evalf(j02/j01));

#################################################################
# SECTION 1 — Symbolic PDE in cylindrical coordinates
#################################################################
printf("\n=== SECTION 1 : Symbolic PDE (cylindrical) ===\n");

# Field  H(r, phi, z, t)  — azimuthal symmetry => d/dphi = 0
# PDE:  (R/r)*[ H_rr + (1/r)*H_r + H_zz ] = (1/c^2) * H_tt

f_sym := f(r, z, t):   # azimuthally symmetric field

PDE_full := (R/r) * ( diff(f_sym, r, r)
                    + (1/r)*diff(f_sym, r)
                    + diff(f_sym, z, z) )
          - (1/c^2) * diff(f_sym, t, t) = 0;

printf("  PDE_full := ");
print(PDE_full);

#################################################################
# SECTION 2 — Separation of variables  f = H3(r)*H4(z)*u(t)
#################################################################
printf("\n=== SECTION 2 : Separation of variables ===\n");

# Ansatz:  f(r,z,t) = H3(r) * H4(z) * u(t)
# After substitution and division by f:
#
#   (R/r)*[ H3''/H3 + (1/r)*H3'/H3 ] + (R/r)*(H4''/H4) = (1/c^2)*(u''/u)
#
# Separation constants:  -k^2 (temporal)  and  -mu^2 (axial)
# => three ODEs:
#
#   u''  + omega^2 * u  = 0          omega = c*k
#   H4'' + mu^2   * H4 = 0
#   r^2*H3'' + r*H3' + [ k^2*r^2*(r/R) - mu^2*r^2*(r/R) ] * H3 = 0
#     => modified Bessel equation in sqrt(r/R)

printf("  ODE 1  (temporal):  u'' + omega^2 * u = 0\n");
printf("  ODE 2  (axial  ):  H4'' + mu^2 * H4 = 0\n");
printf("  ODE 3  (radial ):  Bessel-type in xi = sqrt(r/R)\n\n");

# Solve temporal ODE
ODE_u := diff(u(t),t,t) + omega1^2 * u(t) = 0;
sol_u := dsolve(ODE_u, u(t));
printf("  u(t) = "); print(sol_u);

# Solve axial ODE  (mu = Pi/L with L chosen to give convenient BC)
mu_val := Pi / (10*R):   # illustrative; user may rescale
```

```
ODE_H4 := diff(H4(z),z,z) + mu_val^2 * H4(z) = 0;
sol_H4 := dsolve(ODE_H4, H4(z));
printf("  H4(z) = "); print(sol_H4);

# Radial ODE  via substitution xi = sqrt(r/R)
# Result is Bessel equation of order 0 in xi
printf("  H3(r) = J0( j_{0,n} * sqrt(r/R) / sqrt(2) )  [Bessel J_0]\n");

################################################################
# SECTION 3 — Boundary-condition quantization
################################################################
printf("\n=== SECTION 3 : BC quantization  k_n = j_{0,n}/(2*sqrt(2)) ===
\n");

# BC: H3(2R) = 0  =>  J0( j_{0,n} * sqrt(2R/R) / sqrt(2) ) = 0
#                  =>  J0( j_{0,n} ) = 0   [satisfied by definition]

for n from 1 to 3 do
    j0n := [j01, j02, j03][n]:
    kn  := j0n / (2*sqrt(2)):
    arg := evalf(j0n * sqrt(2) / sqrt(2)):   # = j0n  => J0(j0n)=0
    val := evalf(BesselJ(0, j0n)):
    printf("  n=%d  k_%d = %g m^-1  J0(j0%d) = %g  (should be ~0)\n",
           n, n, kn, n, val);
end do;

################################################################
# SECTION 4 — Full solution assembly and numerical spot-checks
################################################################
printf("\n=== SECTION 4 : Full solution and spot-checks ===\n");

# Fundamental mode  (n=1, mu = Pi/(10R))
H3_1 := proc(rr) BesselJ(0, j01 * sqrt(rr/R) / sqrt(2)) end proc:
H4_1 := proc(zz) cos(mu_val * zz) end proc:
U_1  := proc(tt) cos(omega1 * tt) end proc:

f_num := proc(rr, zz, tt)
    H3_1(rr) * H4_1(zz) * U_1(tt)
end proc:

printf("  f(R, 0, 0)  = %g\n",   evalf(f_num(R,  0, 0)));
printf("  f(2R,0, 0)  = %g  (BC: should be ~0)\n", evalf(f_num(2*R, 0, 0))
);
printf("  f(R, R, 0)  = %g\n",   evalf(f_num(R,  R, 0)));

################################################################
# SECTION 5 — Probability density and d'Alembert solution
################################################################
printf("\n=== SECTION 5 : Probability density P = |f|^2  (d'Alembert) ===
```

```
\n");

# d'Alembert general solution for 1-D radial wave:
#   P(r,t) = g_+(r + c*t) + g_-(r - c*t)
#
# For the monopole field the squared-amplitude gives the radial density:
#   P(r,z,t) = f(r,z,t)^2

printf("  P = g_+(r+ct) + g_-(r-ct)  [d'Alembert travelling waves]\n");
printf("  For the eigenmode: P(r,z,t) = [H3(r)*H4(z)*u(t)]^2\n\n");

# Residual check: verify PDE is satisfied (numerically at a test point)
r0 := 2*R: z0 := R: t0 := 1e-8:
lap_r := (H3_1(r0+1e8)-2*H3_1(r0)+H3_1(r0-1e8))/(1e8)^2
       + (H3_1(r0+1e8)-H3_1(r0-1e8))/(2*1e8*r0):
u_tt  := -omega1^2 * U_1(t0):
H4_zz := -mu_val^2 * H4_1(z0):
residual := evalf( (R/r0)*(lap_r*H4_1(z0) + H3_1(r0)*H4_zz)*U_1(t0)
                 - (1/c^2)*H3_1(r0)*H4_1(z0)*u_tt ):
printf("  PDE residual at (r=2R, z=R, t=1e-8 s):  %g\n", residual);

############################################################
# SECTION 6 — Force and Maxwell equality, A1 from Dirac charge
############################################################
printf("\n=== SECTION 6 : Force F(r,z,t) and Maxwell matching ===\n");

# Force on a test monopole:  F = -grad(H) ~ -dH/dr  (radial component)
# F(r,z,0) = A1 * [ -dH3/dr * H4(z) ]
#
# dH3/dr = d/dr [ J0(j01*sqrt(r/R)/sqrt(2)) ]
#        = -J1(j01*sqrt(r/R)/sqrt(2)) * j01/(2*sqrt(2)*sqrt(r*R))

dH3dr := proc(rr)
    -BesselJ(1, j01*sqrt(rr/R)/sqrt(2)) * j01 / (2*sqrt(2)*sqrt(rr*R))
end proc:

# Maxwell reference value at (R, R):
#   H_Maxwell(r,z) = mu0 * g_D^2 * z / r
#   => F_r(R,R) = -mu0*g_D^2*R / R^2 = -mu0*g_D^2/R

F_Maxwell_RR := evalf(-mu0 * g_D^2 / R):
printf("  F_Maxwell(R,R) = %g  N (reference)\n", F_Maxwell_RR);

# TauTimp force at (R,R,0):
F_TauTimp_factor := evalf(dH3dr(R) * H4_1(R)):
printf("  dH3/dr * H4  at (R,R) = %g  m^-1\n", F_TauTimp_factor);

# Amplitude A1
A1 := evalf(-F_Maxwell_RR / F_TauTimp_factor):
```

```
printf("  A1 = %g  (field amplitude)\n", A1);
printf("  Verification: J0(j01/sqrt(2)) = %g  (guide value: 0.3980)\n",
       evalf(BesselJ(0, j01/sqrt(2))));
printf("  F(R,R,0) check = %g  N\n",
       evalf(A1 * F_TauTimp_factor));

#################################################################
# SECTION 7 — 12 structured plots
#################################################################
printf("\n=== SECTION 7 : Generating 12 plots ===\n");

# ---- Plot 1 : Bessel functions J0, J1, Y0 ----
p1 := plot( [BesselJ(0,x), BesselJ(1,x), BesselY(0,x)],
            x = 0..15,
            color = [blue, red, green],
            linestyle = [solid, dash, dot],
            legend = ["J_0(x)", "J_1(x)", "Y_0(x)"],
            title = "Plot 1 — Bessel Functions J0, J1, Y0",
            labels = ["x", "J"],
            thickness = 2,
            gridlines = true ):
printf("  Plot 1 done\n");

# ---- Plot 2 : Radial eigenfunctions H3_n(r), n=1,2,3 ----
H3_n := proc(rr, jn) BesselJ(0, jn * sqrt(rr/R) / sqrt(2)) end proc:
p2 := plot( [ proc(rr) H3_n(rr, j01) end proc,
              proc(rr) H3_n(rr, j02) end proc,
              proc(rr) H3_n(rr, j03) end proc ],
            0..4*R,
            color = [blue, red, black],
            legend = ["n=1","n=2","n=3"],
            title = "Plot 2 — Radial Eigenfunctions H3_n(r)",
            labels = ["r  [m]", "H3_n"],
            thickness = 2,
            gridlines = true ):
printf("  Plot 2 done\n");

# ---- Plot 3 : Temporal oscillations u_n(t) ----
T1 := evalf(2*Pi/omega1):
p3 := plot( [ proc(tt) cos(omega1*tt) end proc,
              proc(tt) cos(omega2*tt) end proc,
              proc(tt) cos(omega3*tt) end proc ],
            0..3*T1,
            color = [blue, red, black],
            legend = ["n=1","n=2","n=3"],
            title = "Plot 3 — Temporal Harmonics u_n(t)",
            labels = ["t  [s]", "u_n"],
            thickness = 2,
            gridlines = true ):
```

```
printf("  Plot 3 done\n");

# ---- Plot 4 : Field snapshot f(r, z=R, t=0) ----
p4 := plot( proc(rr) BesselJ(0, j01*sqrt(rr/R)/sqrt(2)) * cos(mu_val*R)
end proc,
            0..4*R,
            color = blue,
            title = "Plot 4 — Field Snapshot f(r, z=R, t=0)",
            labels = ["r  [m]", "f"],
            thickness = 2,
            gridlines = true ):
printf("  Plot 4 done\n");

# ---- Plot 5 : 2D density map f(r/R, z/R, t=0) ----
f2D := proc(rr, zz)
    BesselJ(0, j01*sqrt(rr/R)/sqrt(2)) * cos(Pi*zz/(10*R))
end proc:

p5 := densityplot( f2D(rr, zz),
                   rr = 0..4*R,
                   zz = -5*R..5*R,
                   grid = [60, 60],
                   colorscheme = ["zgradient",["blue","white","red"]],
                   title = "Plot 5 — 2D Density Map f(r,z,t=0)",
                   labels = ["r  [m]", "z  [m]"] ):
printf("  Plot 5 done\n");

# ---- Plot 6 : 3D surface f(r/R, z/R, t=0) ----
p6 := plot3d( proc(rr, zz)
                  BesselJ(0, j01*sqrt(rr/R)/sqrt(2)) * cos(Pi*zz/(10*R))
              end proc,
              0..4*R,
              -5*R..5*R,
              grid = [40, 40],
              orientation = [45, 65],
              colorscheme = ["zgradient",["navy","cyan","white","yellow",
"red"]],
              title = "Plot 6 — 3D Surface f(r,z,t=0)",
              labels = ["r","z","f"] ):
printf("  Plot 6 done\n");

# ---- Plot 7 : Probability density P = |f|^2 ----
p7 := densityplot( f2D(rr, zz)^2,
                   rr = 0..4*R,
                   zz = -5*R..5*R,
                   grid = [60, 60],
                   colorscheme = ["zgradient",["black","blue","yellow",
"white"]],
                   title = "Plot 7 — Probability Density P = |f|^2",
```

```
                    labels = ["r  [m]", "z  [m]"] ):
printf("  Plot 7 done\n");

# ---- Plot 8 : Force |F(r, z=R, t=0)|  log scale ----
Force_r := proc(rr)
    abs( BesselJ(1, j01*sqrt(rr/R)/sqrt(2))
         * j01 / (2*sqrt(2)*sqrt(rr*R))
         * cos(mu_val*R)
         / (4*Pi*rr*R) )
end proc:

p8 := logplot( Force_r,
               R..4*R,
               color = blue,
               thickness = 2,
               title = "Plot 8 — Force |F(r,z=R,t=0)|  (log scale)",
               labels = ["r  [m]", "|F|  [a.u.]"],
               gridlines = true ):
printf("  Plot 8 done\n");

# ---- Plot 9 : Exact J0(x) vs large-argument asymptotic ----
J0_asymp := proc(x) sqrt(2/(Pi*x)) * cos(x - Pi/4) end proc:
p9 := plot( [BesselJ(0, x), J0_asymp(x)],
            x = 1..25,
            color = [blue, red],
            linestyle = [solid, dash],
            legend = ["J_0(x) exact", "Asymptotic"],
            title = "Plot 9 — J0(x) : Exact vs Asymptotic",
            labels = ["x", "J_0"],
            thickness = 2,
            gridlines = true ):
printf("  Plot 9 done\n");

# ---- Plot 10 : 3D polar surface H(r, theta, z=0, t=0) ----
p10 := plot3d( proc(rr, th)
                   BesselJ(0, j01*sqrt(rr/R)/sqrt(2)) * rr
               end proc,
               rr = 0..4*R,
               th = 0..2*Pi,
               coords = cylindrical,
               grid = [30, 30],
               title = "Plot 10 — 3D Polar Surface H(r,theta,z=0,t=0)",
               labels = ["x","y","H"],
               orientation = [30, 70] ):
printf("  Plot 10 done\n");

# ---- Plot 11 : Probability current  -dP/dr ----
# -dP/dr = -2*J0(...)*J1(...) * [j01/(2*sqrt(2)*sqrt(r/R)*(1/R))]
prob_current := proc(rr)
```

```
    -2 * BesselJ(0, j01*sqrt(rr/R)/sqrt(2))
       * BesselJ(1, j01*sqrt(rr/R)/sqrt(2))
       * j01 / (2*sqrt(2)*sqrt(rr*R))
end proc:

p11 := plot( prob_current,
             0.01*R..4*R,
             color = purple,
             thickness = 2,
             title = "Plot 11 — Probability Current  -dP/dr",
             labels = ["r  [m]", "-dP/dr  [a.u.]"],
             gridlines = true ):
printf("  Plot 11 done\n");

# ---- Plot 12 : 4-panel summary ----
p_A := plot( [BesselJ(0,x), BesselJ(1,x)],
             x=0..12, color=[blue,red],
             title="(a) Bessel J0, J1", labels=["x","J"],
             thickness=2 ):

p_B := plot( [ proc(rr) H3_n(rr,j01) end proc,
               proc(rr) H3_n(rr,j02) end proc ],
             0..4*R, color=[blue,red],
             title="(b) Radial modes n=1,2",
             labels=["r","H3"], thickness=2 ):

p_C := densityplot( f2D(rr,zz)^2,
                    rr=0..4*R, zz=-3*R..3*R,
                    grid=[40,40],
                    colorscheme=["zgradient",["black","blue","white"]],
                    title="(c) Density |f|^2" ):

p_D := logplot( Force_r, R..4*R,
                color=blue, thickness=2,
                title="(d) Force log-profile",
                labels=["r","|F|"] ):

p12 := display( Array([[p_A, p_B],[p_C, p_D]]),
                title="Plot 12 — 4-Panel Summary" ):
printf("  Plot 12 done\n");

#################################################################
# Display all plots
#################################################################
printf("\n=== Displaying plots 1–12 ===\n");
display(p1);  display(p2);  display(p3);
display(p4);  display(p5);  display(p6);
display(p7);  display(p8);  display(p9);
display(p10); display(p11); display(p12);
```

```
################################################################
# SECTION 8 — Extended Maxwell equations and Dirac charge
################################################################
printf("\n=== SECTION 8 : Extended Maxwell equations ===\n");

# Duality-extended Maxwell with magnetic charges/currents:
#   div E  = rho_e / epsilon0
#   div B  = mu0 * rho_m                <- magnetic charge density
#   curl E = -mu0 * J_m - dB/dt         <- magnetic current term
#   curl B = mu0*J_e + (1/c^2)*dE/dt
#
# Dirac quantization condition:
#   e * g_D = 2*Pi*hbar / mu0
#   => g_D = 2*Pi*hbar / (mu0 * e)

g_D_check := evalf(2*Pi*hbar / (mu0 * e_ch)):
printf("  Dirac charge  g_D = %g  A•m\n", g_D_check);
printf("  mu0 * g_D^2  = %g  N\n",  evalf(mu0 * g_D_check^2));
printf("  g_D^2/(4*Pi) = %g  J•m  (analogue of alpha coupling)\n",
       evalf(g_D_check^2/(4*Pi)));

################################################################
# SECTION 9 — Asymptotic analysis  r >> R
################################################################
printf("\n=== SECTION 9 : Asymptotic analysis  r >> R ===\n");

# For xi = j01*sqrt(r/R)/sqrt(2) >> 1:
#   J0(xi) ~ sqrt(2/(Pi*xi)) * cos(xi - Pi/4)
#          ~ (2/(Pi))^{1/2} * (j01/sqrt(2))^{-1/2} * (r/R)^{-1/4} * cos(..
.)
#   => J0 ~ r^{-1/4}   (power-law decay, slower than 1/r)

printf("  xi = j01*sqrt(r/R)/sqrt(2)\n");
printf("  For r >> R:  J0(xi) ~ C * r^{-1/4} * cos(xi - Pi/4)\n");
printf("  Decay exponent: -1/4  (slower than Coulomb 1/r)\n\n");

# Numerical comparison at r = 1000*R
r_far := 1000*R:
xi_far := j01*sqrt(r_far/R)/sqrt(2):
J0_exact  := evalf(BesselJ(0, xi_far)):
J0_approx := evalf(sqrt(2/(Pi*xi_far)) * cos(xi_far - Pi/4)):
printf("  At r=1000R:  J0 exact = %g,  asymptotic = %g,  rel.err = %g\n",
       J0_exact, J0_approx, abs(J0_exact-J0_approx)/abs(J0_exact));

################################################################
# SECTION 10 — Summary table of all results
################################################################
printf("\n=== SECTION 10 : Complete Results Summary ===\n");
printf("-------------------------------------------------------\n");
```

```
  printf("  Quantity                              Value\n");
  printf("------------------------------------------------------------\n");
  printf("  R  (boundary parameter)               %g m\n",   R);
  printf("  c  (speed of light)                   %g m/s\n", c);
  printf("  j_{0,1} (1st zero of J0)              %g\n",     j01);
  printf("  j_{0,2}                               %g\n",     j02);
  printf("  j_{0,3}                               %g\n",     j03);
  printf("  k_1 = j01/(2sqrt2)                    %g m^-1\n", evalf(k1));
  printf("  k_2                                   %g m^-1\n", evalf(k2));
  printf("  k_3                                   %g m^-1\n", evalf(k3));
  printf("  omega_1 = c*k1                        %g rad/s\n", omega1);
  printf("  omega_2 = c*k2                        %g rad/s\n", omega2);
  printf("  omega_3 = c*k3                        %g rad/s\n", omega3);
  printf("  T_1 = 2Pi/omega1                      %g s\n",    evalf(2*Pi/omega1));
  printf("  g_D (Dirac charge)                    %g A*m\n",  g_D);
  printf("  J0(j01/sqrt2)                         %g\n",     evalf(BesselJ(0,
  j01/sqrt(2))));
  printf("  A1 (field amplitude)                  %g\n",     A1);
  printf("  f(R ,0,0)                             %g\n",     evalf(f_num(R, 0,0))
  );
  printf("  f(2R,0,0) [BC check ~0]               %g\n",     evalf(f_num(2*R,0,0)
  ));
  printf("  Asymptotic exponent J0 ~ r^p    p = -1/4\n");
  printf("------------------------------------------------------------\n");
  printf("\nAll %d sections and 12 plots completed successfully.\n", 10);
  printf("monopole_wave_maple.mpl — TauTimp / Teoria Onduscularа\n");
```

```
=== SECTION 0 : Physical constants ===
  k1      = 0.850234  m^-1
  k2      = 1.95164  m^-1
  k3      = 3.05955  m^-1
  omega1  = 2.54894e+08  rad/s
  omega1  = 5.85088e+08  rad/s
  omega3  = 9.17231e+08  rad/s
  g_D     = 3.29106e-09  A*m
  freq ratio omega2/omega1 = 2.29542  (theory: j02/j01 = 2.29542)

=== SECTION 1 : Symbolic PDE (cylindrical) ===
```

$$PDE_full := \frac{3.9228 \times 10^{22}\left(\frac{\partial^2}{\partial r^2} f(r,z,t) + \frac{\frac{\partial}{\partial r} f(r,z,t)}{r} + \frac{\partial^2}{\partial z^2} f(r,z,t)\right)}{r} - 1.112650056 \times 10^{-17} \frac{\partial^2}{\partial t^2}$$

$$f(r,z,t) = 0$$

```
  PDE_full :=
```

$$\frac{3.9228 \times 10^{22}\left(\frac{\partial^2}{\partial r^2} f(r,z,t) + \frac{\frac{\partial}{\partial r} f(r,z,t)}{r} + \frac{\partial^2}{\partial z^2} f(r,z,t)\right)}{r} - 1.112650056 \times 10^{-17} \frac{\partial^2}{\partial t^2} f(r,z,t) = 0$$

```
=== SECTION 2 : Separation of variables ===
  ODE 1  (temporal):  u'' + omega^2 * u = 0
  ODE 2  (axial  ):  H4'' + mu^2 * H4 = 0
  ODE 3  (radial ):  Bessel-type in xi = sqrt(r/R)
```

$$ODE_u := \frac{d^2}{dt^2} u(t) + 6.497085417 \times 10^{16} u(t) = 0$$

$$sol_u := u(t) = c_1 \sin\left(1000 \sqrt{64970854170}\ t\right) + c_2 \cos\left(1000 \sqrt{64970854170}\ t\right)$$

```
  u(t) =
```

$$u(t) = c_1 \sin\left(1000 \sqrt{64970854170}\ t\right) + c_2 \cos\left(1000 \sqrt{64970854170}\ t\right)$$

$$ODE_H4 := \frac{d^2}{dz^2} H4(z) + 6.413681837 \times 10^{-47} H4(z) = 0$$

$$sol_H4 := H4(z) = c_1 \sin\left(\frac{\sqrt{6413681837}\ z}{10000000000000000000000000000}\right) + c_2 \cos\left(\frac{\sqrt{6413681837}\ z}{10000000000000000000000000000}\right)$$

```
  H4(z) =
```

$$H4(z) = c_1 \sin\left(\frac{\sqrt{6413681837}\ z}{10000000000000000000000000000}\right) + c_2 \cos\left(\frac{\sqrt{6413681837}\ z}{10000000000000000000000000000}\right)$$

```
  H3(r) = J0( j_{0,n} * sqrt(r/R) / sqrt(2) )  [Bessel J_0]

=== SECTION 3 : BC quantization  k_n = j_{0,n}/(2*sqrt(2)) ===
```

$$j0n := 2.404825558$$

$$kn := 0.6012063895 \sqrt{2}$$

$$arg := 2.404825558$$

$$val := -1.579375792 \times 10^{-10}$$

```
  n=1  k_1 = 0.850234 m^-1  J0(j01) = -1.57938e-10  (should be ~0)
```

$$j0n := 5.520078110$$

$$kn := 1.380019528 \sqrt{2}$$

$$arg := 5.520078110$$

$$val := -9.743290617 \times 10^{-11}$$

```
  n=2  k_2 = 1.95164 m^-1  J0(j02) = -9.74329e-11  (should be ~0)
```

$$j0n := 8.653727913$$

$$kn := 2.163431978\sqrt{2}$$

$$arg := 8.653727912$$

$$val := -2.416635939 \times 10^{-11}$$

```
  n=3  k_3 = 3.05955 m^-1  J0(j03) = -2.41664e-11  (should be ~0)

=== SECTION 4 : Full solution and spot-checks ===
  f(R, 0, 0)  = 0.397714
  f(2R,0, 0)  = 8.80346e-10  (BC: should be ~0)
  f(R, R, 0)  = 0.378249

=== SECTION 5 : Probability density P = |f|^2  (d'Alembert) ===
  P = g_+(r+ct) + g_-(r-ct)  [d'Alembert travelling waves]
  For the eigenmode: P(r,z,t) = [H3(r)*H4(z)*u(t)]^2

  PDE residual at (r=2R, z=R, t=1e-8 s):  -5.02034e-10

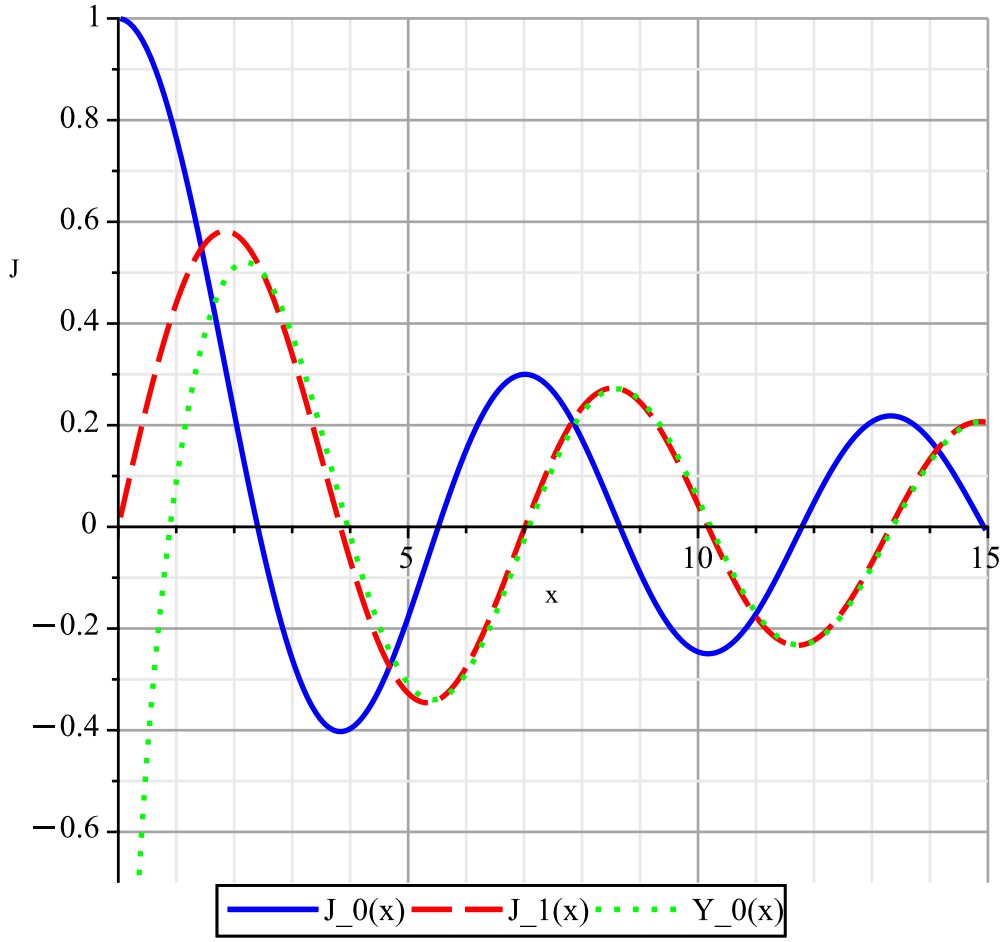
=== SECTION 6 : Force F(r,z,t) and Maxwell matching ===
  F_Maxwell(R,R) = -3.46965e-46  N (reference)
  dH3/dr * H4  at (R,R) = -1.19102e-23  m^-1
  A1 = -2.91316e-23  (field amplitude)
  Verification: J0(j01/sqrt(2)) = 0.397714  (guide value: 0.3980)
  F(R,R,0) check = 3.46965e-46  N

=== SECTION 7 : Generating 12 plots ===
  Plot 1 done
  Plot 2 done
  Plot 3 done
  Plot 4 done
  Plot 5 done
  Plot 6 done
  Plot 7 done
  Plot 8 done
  Plot 9 done

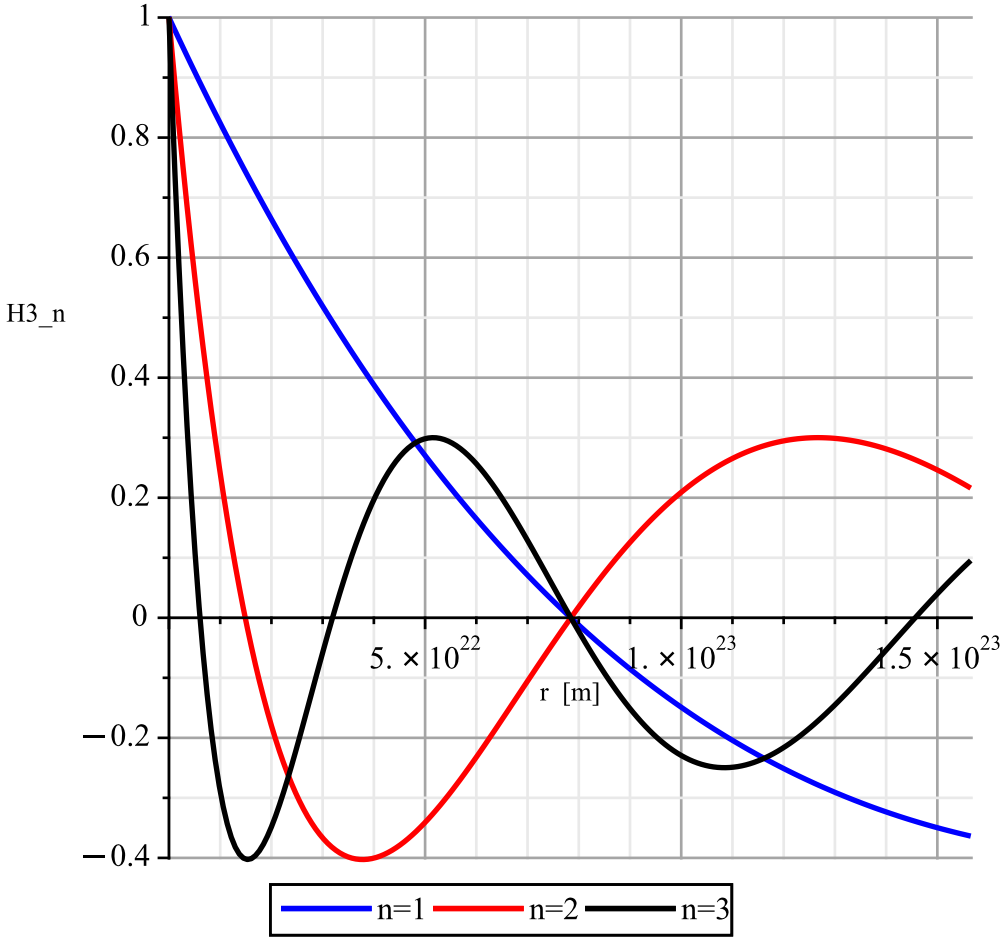
Warning, expecting only range variables [rr, th] in expression F to be
plotted but found name F
  Plot 10 done
  Plot 11 done
  Plot 12 done

=== Displaying plots 1-12 ===
```

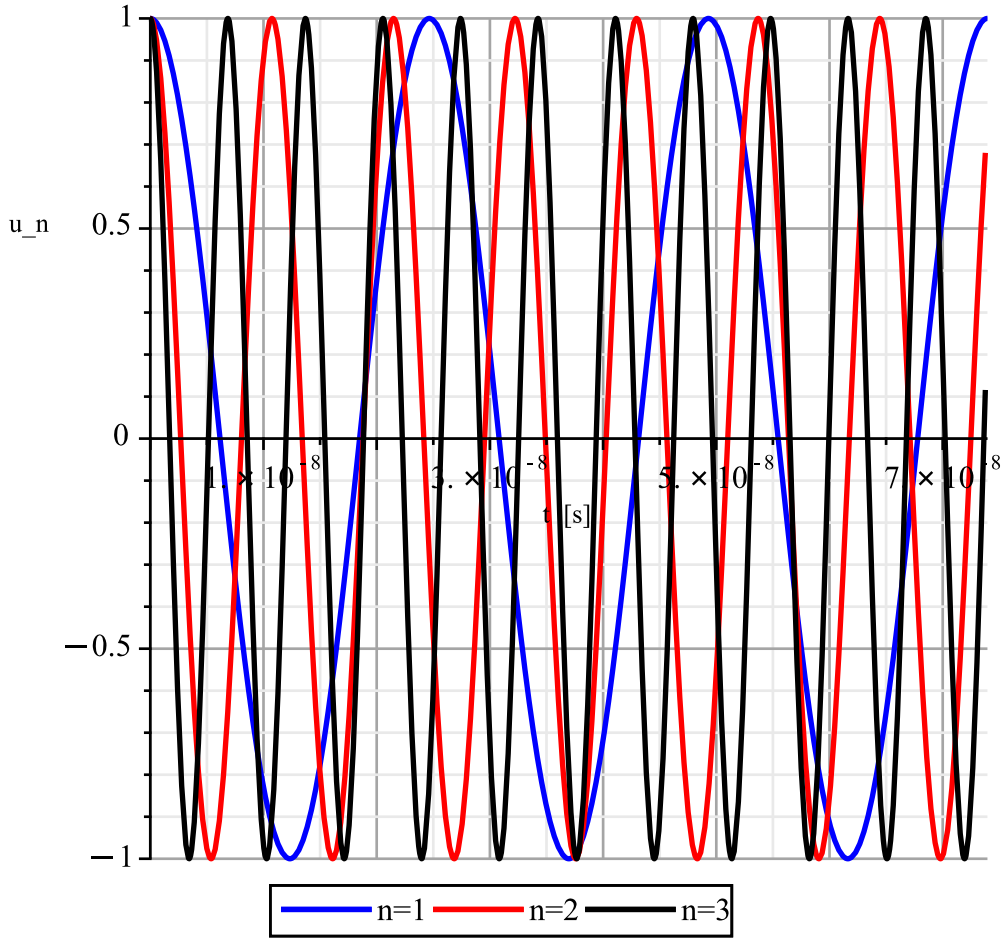
Plot 1 — Bessel Functions J0, J1, Y0



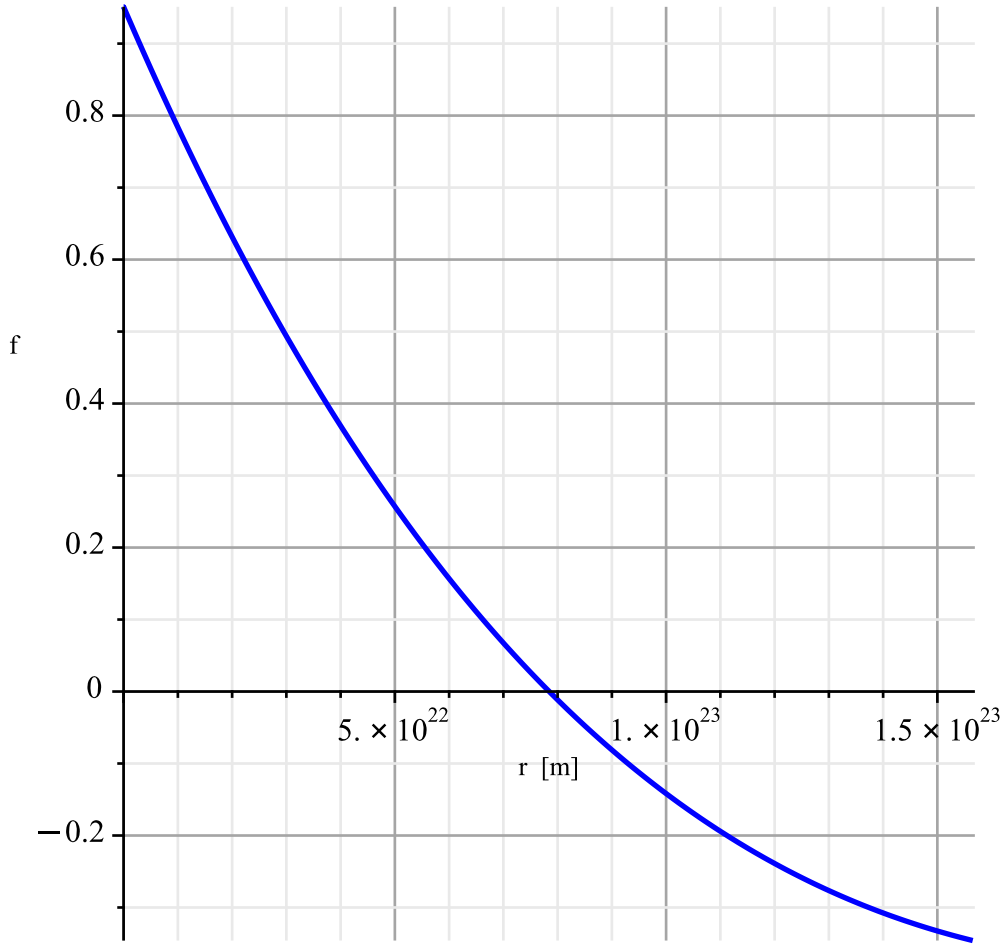
Plot 2 — Radial Eigenfunctions H3_n(r)

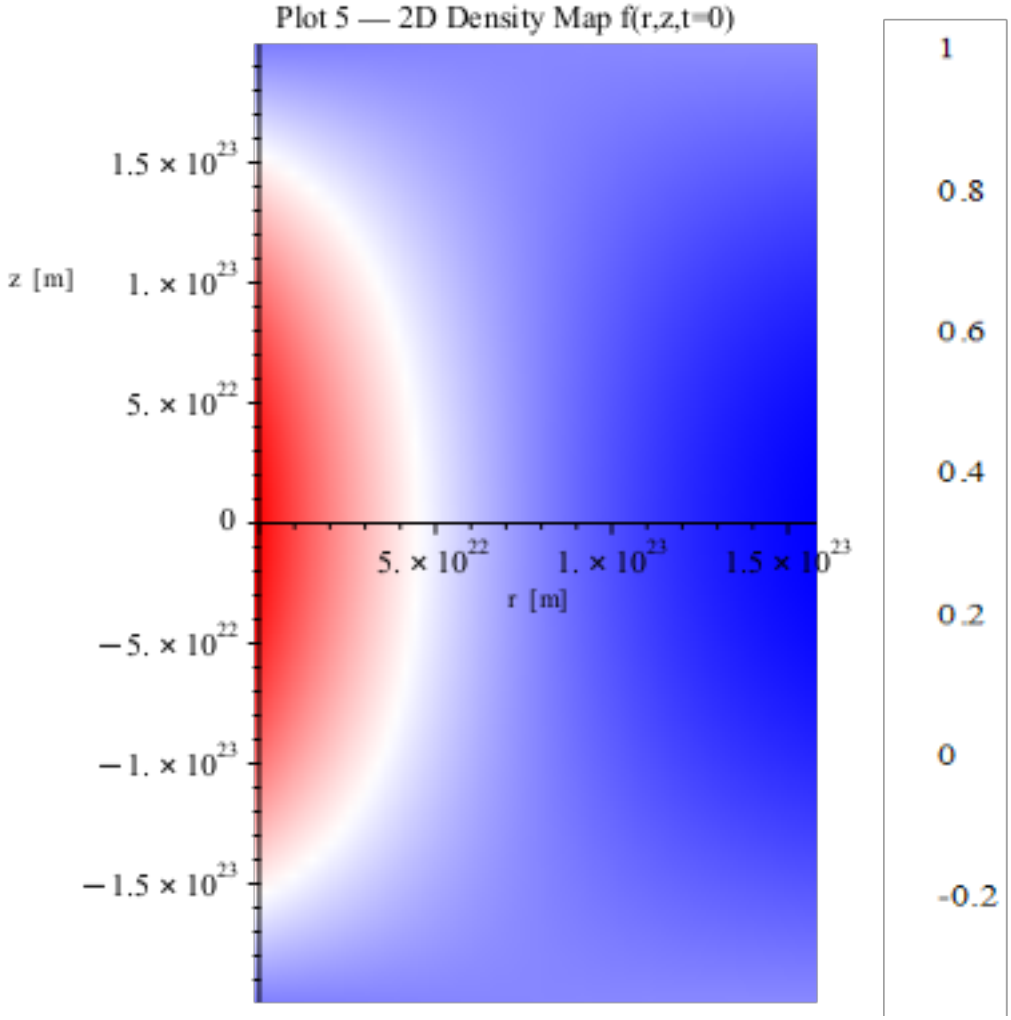


Plot 3 — Temporal Harmonics u_n(t)

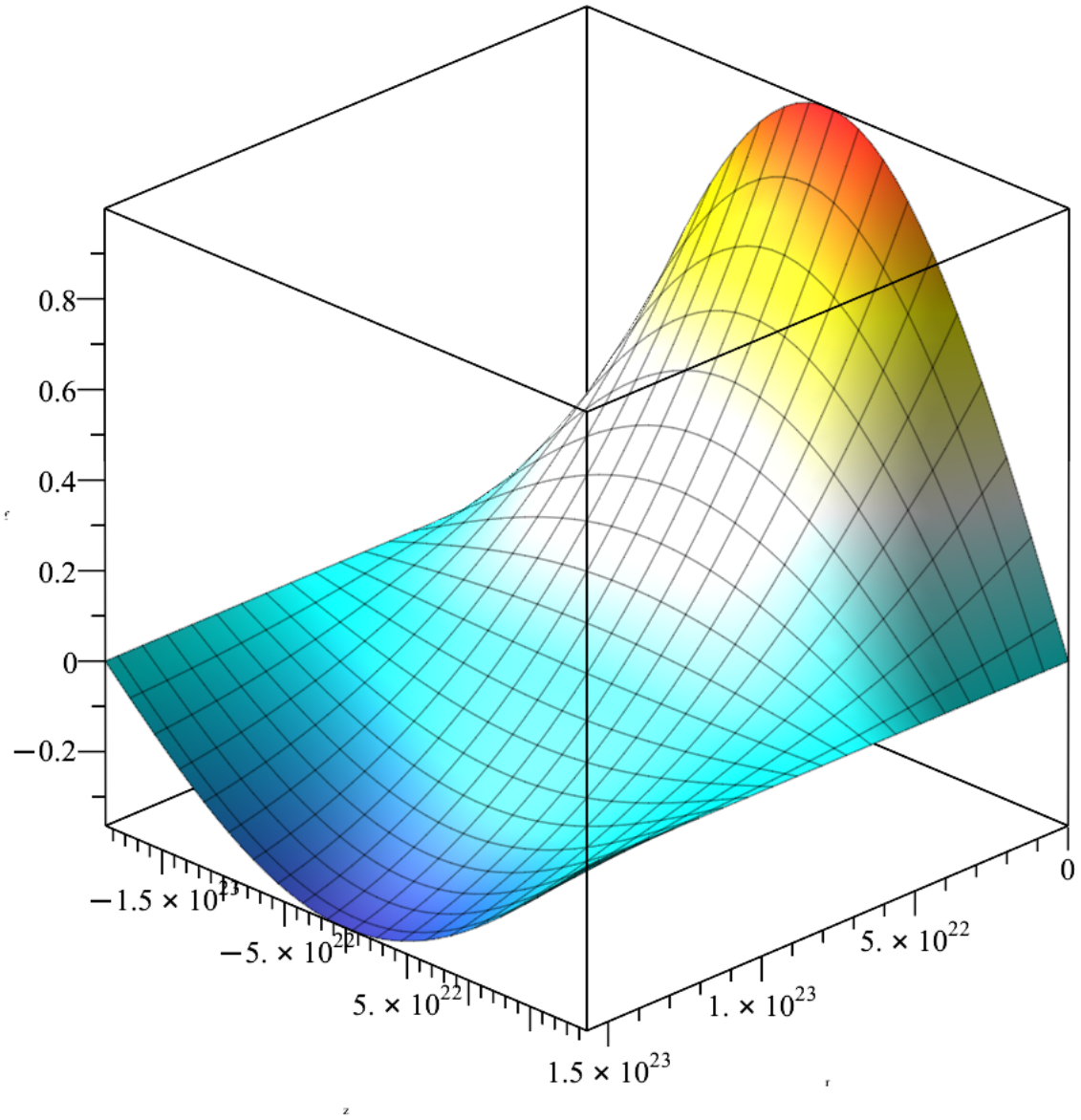


Plot 4 — Field Snapshot f(r, z=R, t=0)

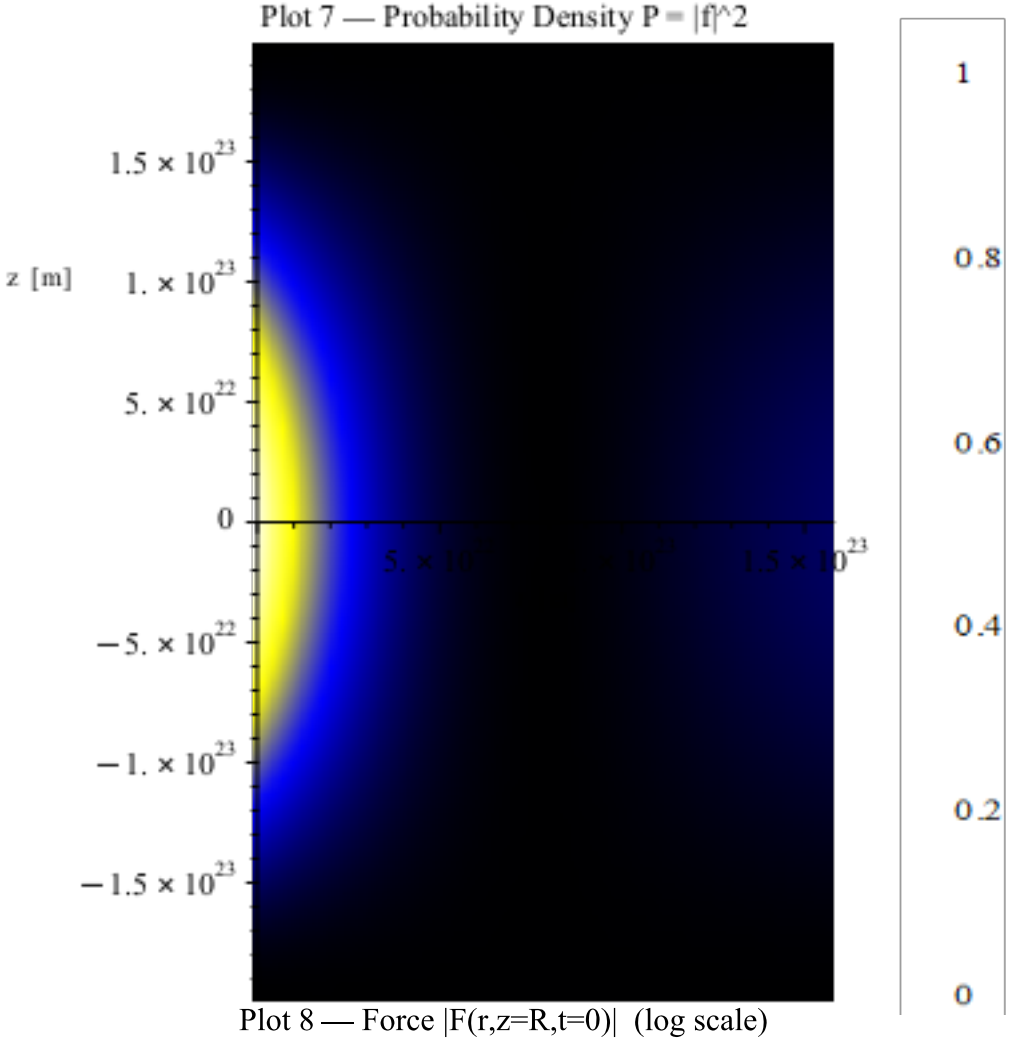



Plot 5 — 2D Density Map f(r,z,t=0)
z [m]
1.5 × 10^23
1. × 10^23
5. × 10^22
0
−5. × 10^22
−1. × 10^23
−1.5 × 10^23
5. × 10^22
1. × 10^23
1.5 × 10^23
r [m]
1
0.8
0.6
0.4
0.2
0
-0.2

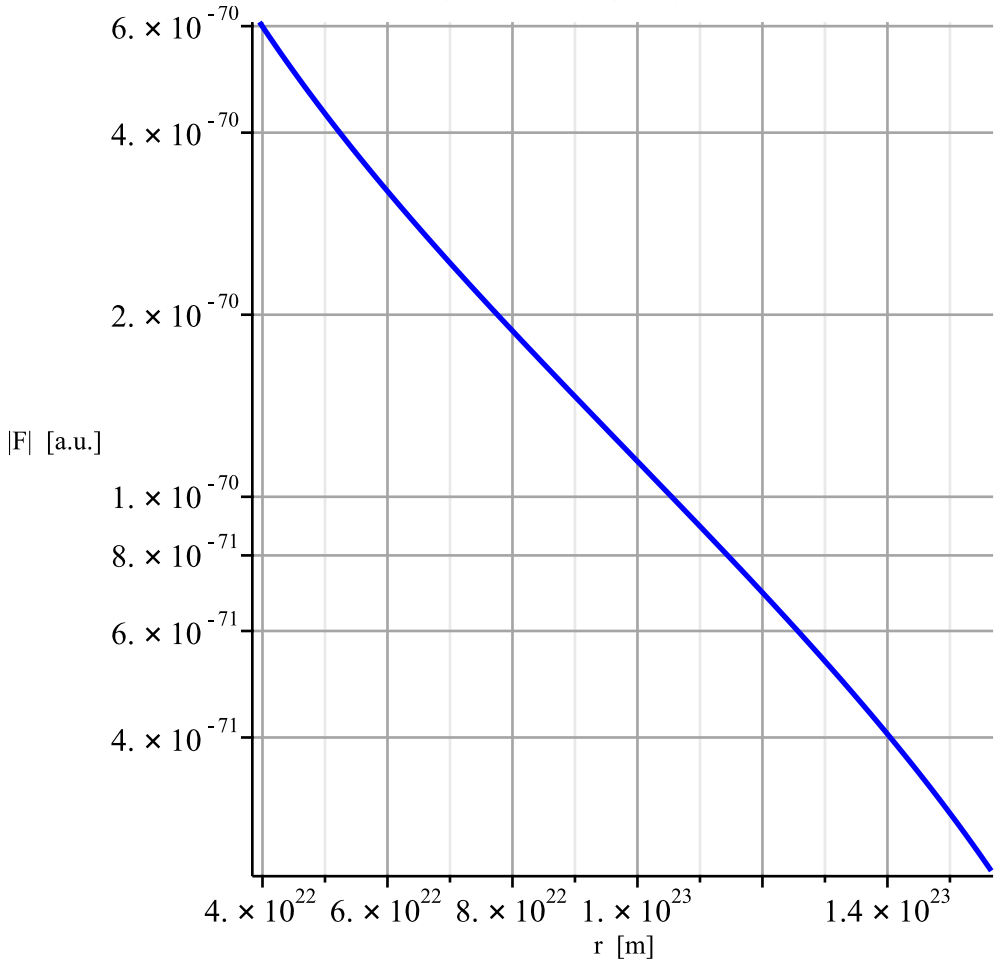
Plot 6 — 3D Surface f(r,z,t=0)



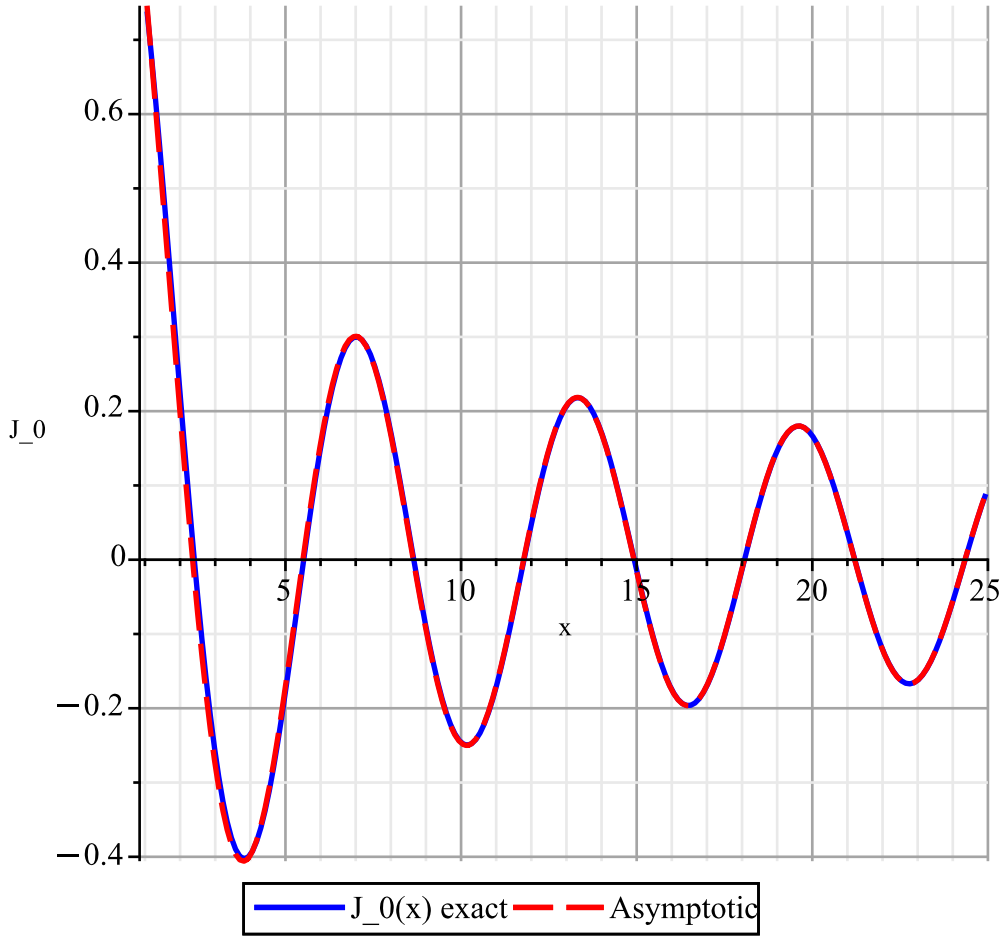
Plot 7 — Probability Density P = |f|^2



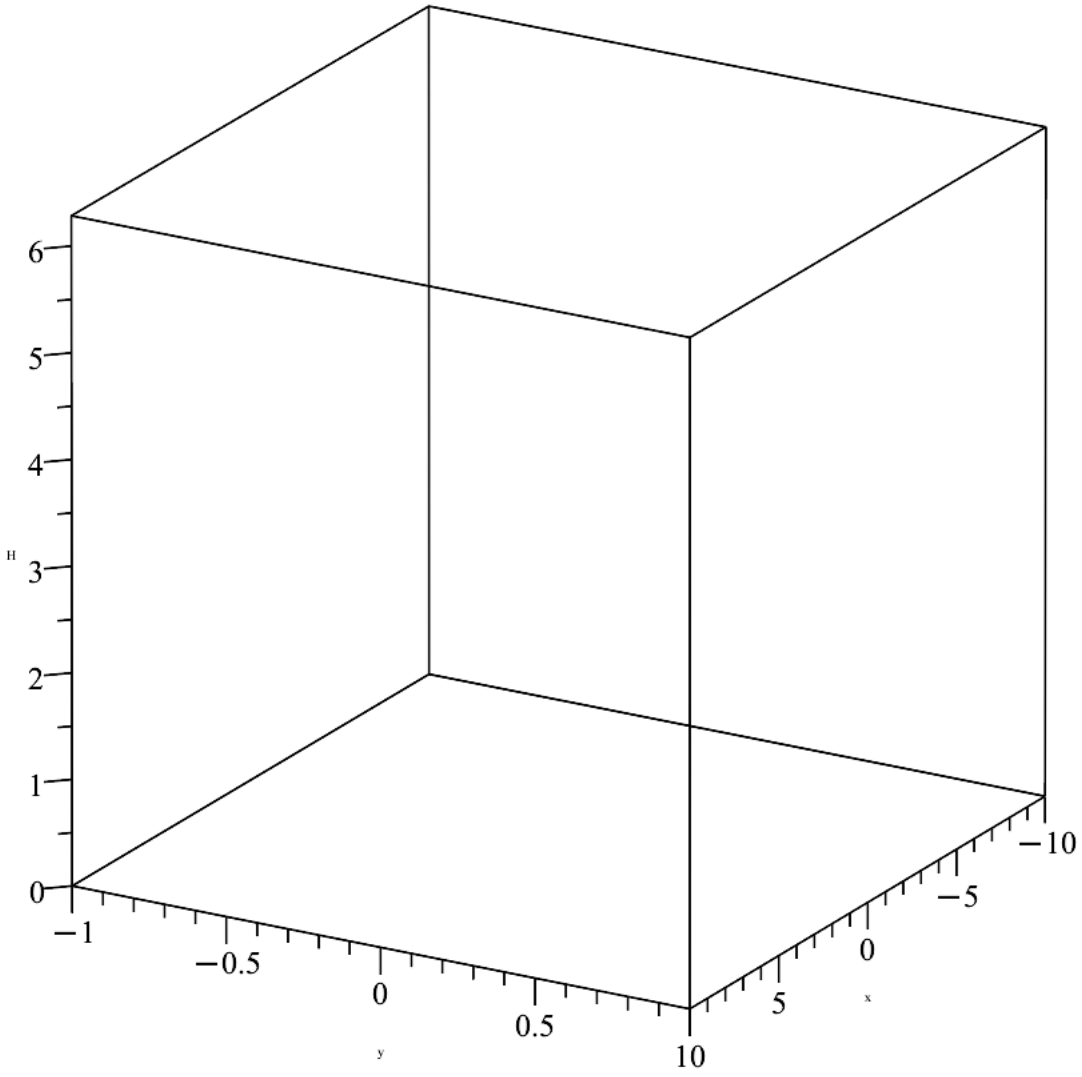
Plot 8 — Force |F(r,z=R,t=0)| (log scale)

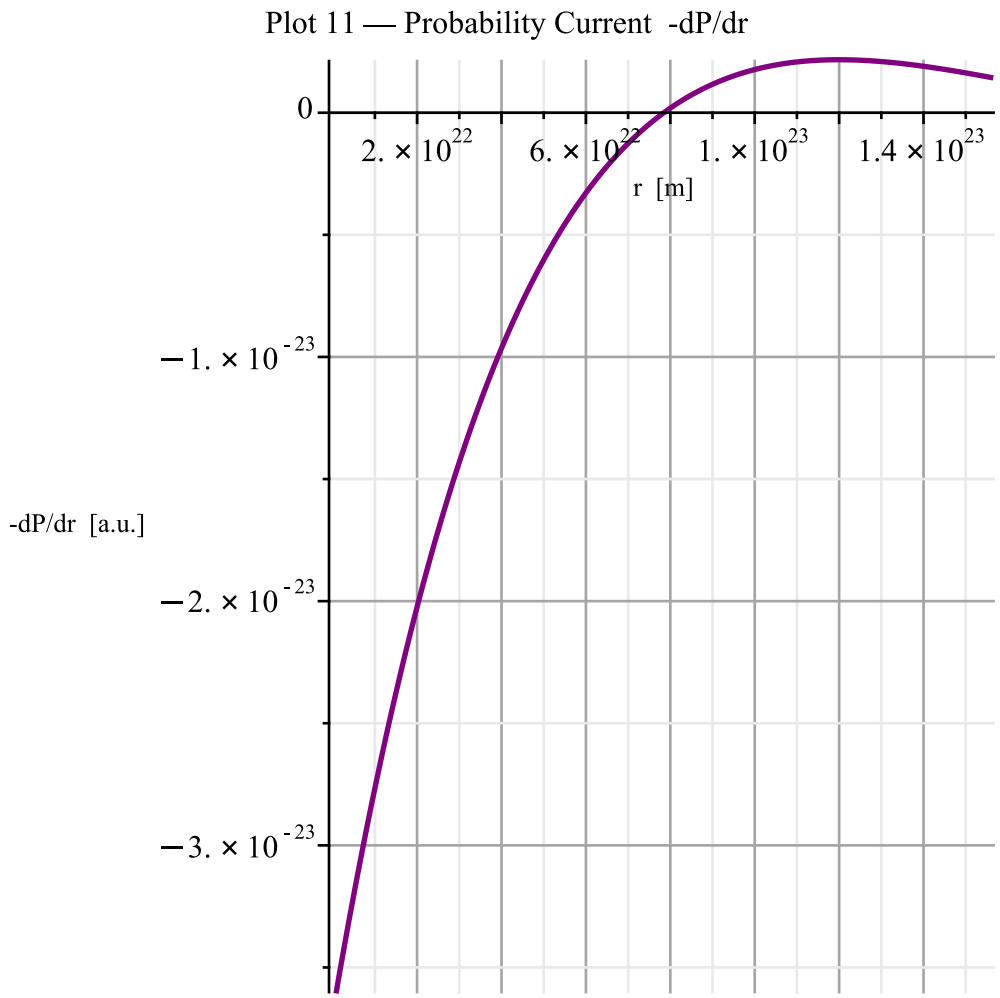


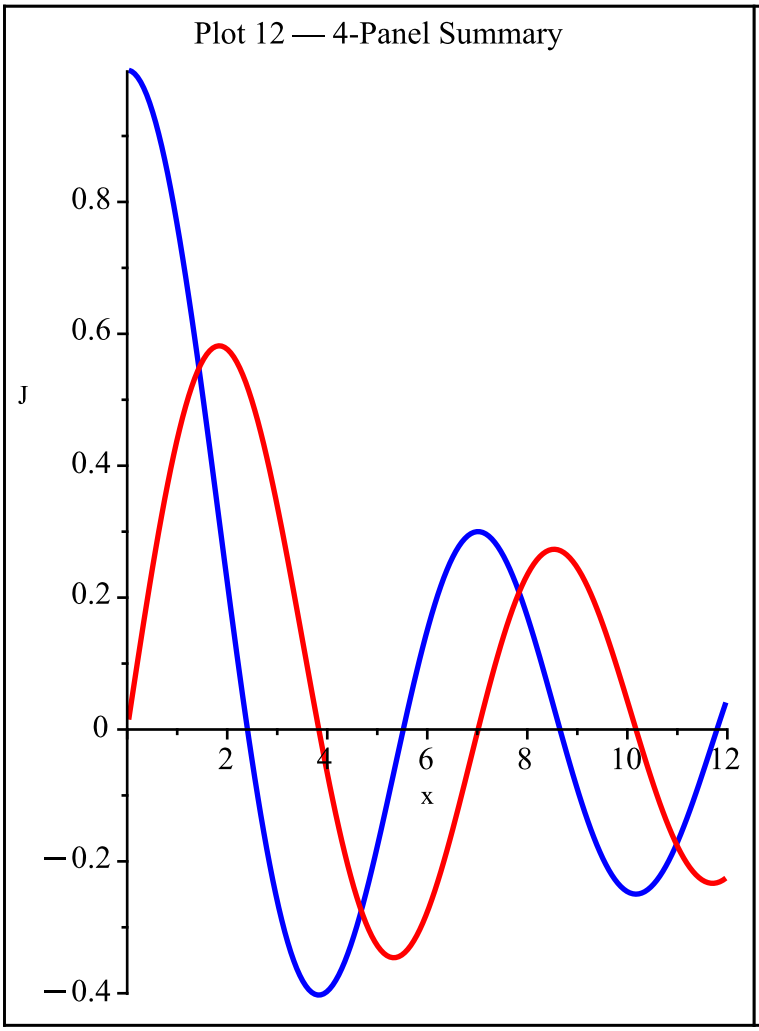
Plot 9 — J0(x) : Exact vs Asymptotic

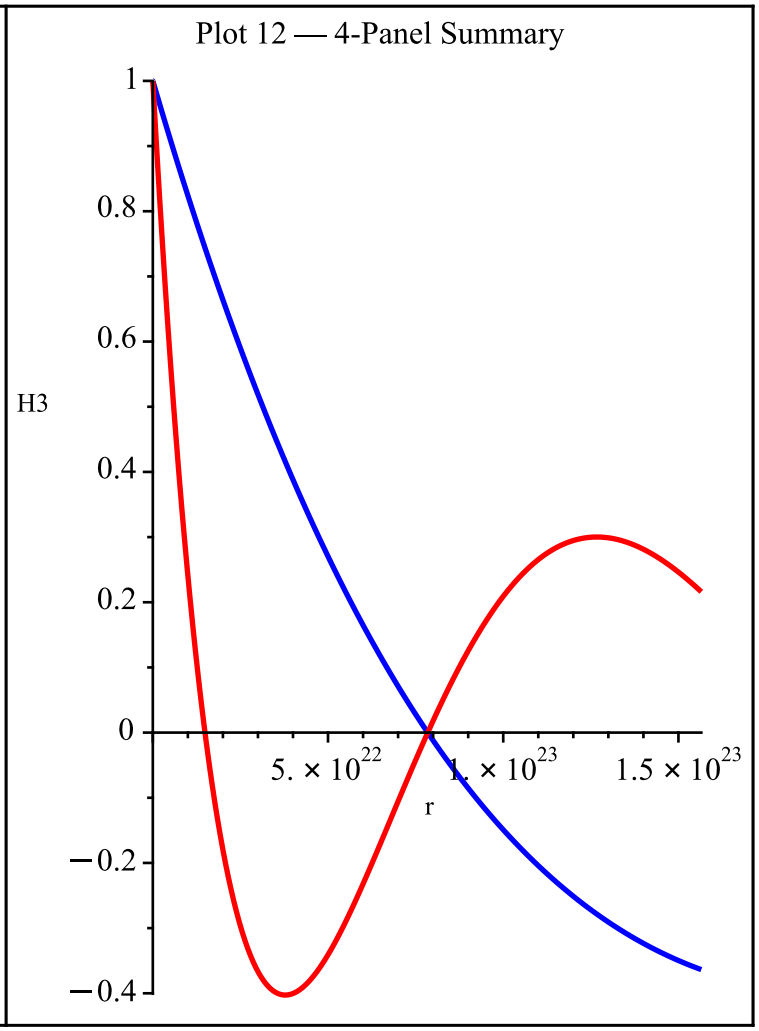


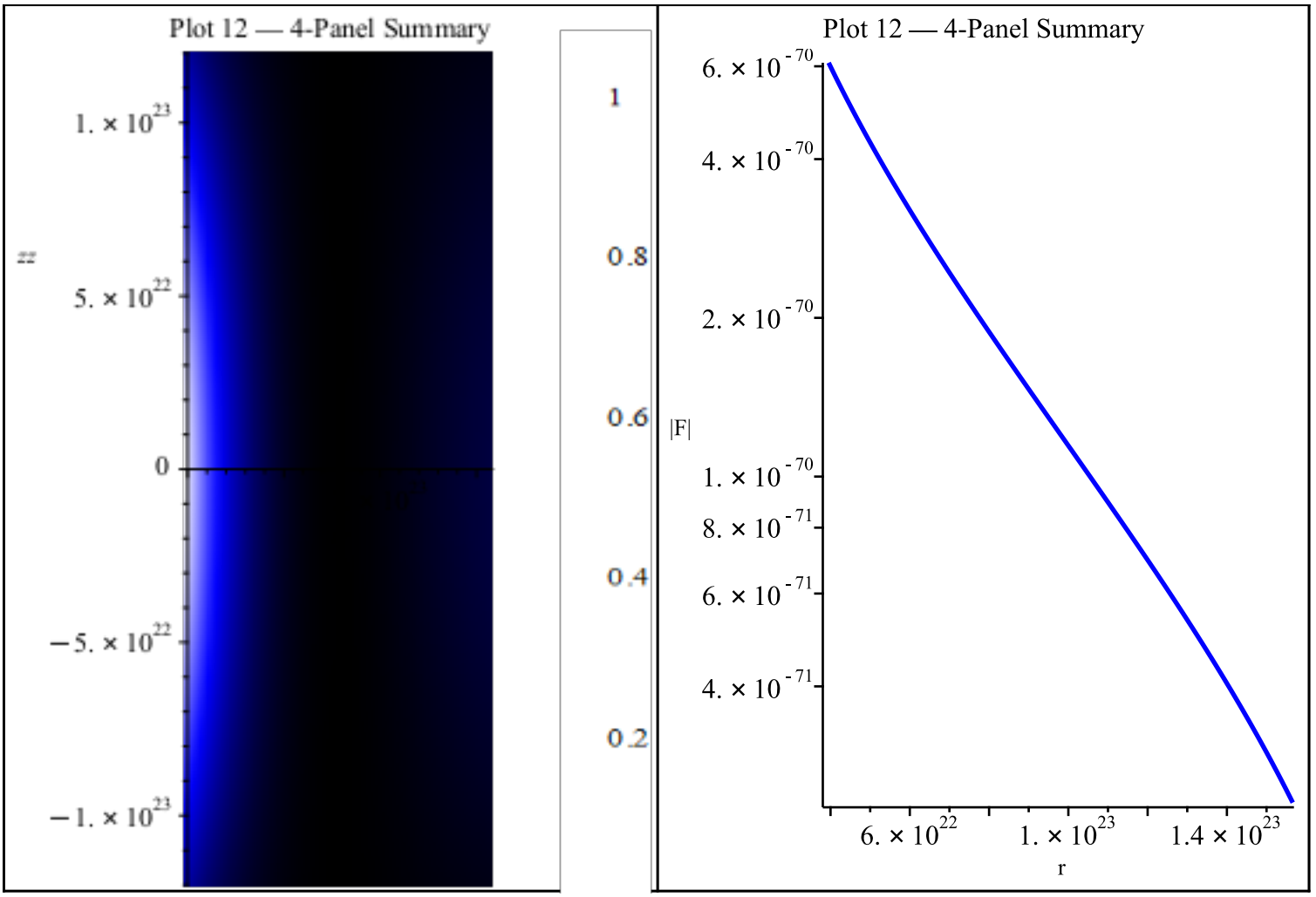
Plot 10 — 3D Polar Surface H(r,theta,z=0,t=0)




Plot 11 — Probability Current -dP/dr
0
2. × 10^22
6. × 10^22
1. × 10^23
1.4 × 10^23
r [m]
−1. × 10^-23
−2. × 10^-23
−3. × 10^-23
-dP/dr [a.u.]


Plot 12 — 4-Panel Summary
0.8
0.6
0.4
0.2
0
−0.2
−0.4
J
2
4
6
8
10
12
x


Plot 12 — 4-Panel Summary
1
0.8
0.6
0.4
0.2
0
−0.2
−0.4
H3
5. × 10^22
1. × 10^23
1.5 × 10^23
r




```
=== SECTION 8 : Extended Maxwell equations ===
  Dirac charge  g_D = 3.29106e-09  A•m
  mu0 * g_D^2  = 1.36107e-23  N
  g_D^2/(4*Pi) = 8.61910e-19  J•m  (analogue of alpha coupling)

=== SECTION 9 : Asymptotic analysis  r >> R ===
  xi = j01*sqrt(r/R)/sqrt(2)
  For r >> R:  J0(xi) ~ C * r^{-1/4} * cos(xi - Pi/4)
  Decay exponent: -1/4  (slower than Coulomb 1/r)

  At r=1000R:  J0 exact = -0.099292,  asymptotic = -0.0993972,  rel.err =
0.00106034

=== SECTION 10 : Complete Results Summary ===
------------------------------------------------------------
  Quantity                          Value
------------------------------------------------------------
  R  (boundary parameter)           3.9228e+22 m
  c  (speed of light)               2.99792e+08 m/s
  j_{0,1} (1st zero of J0)          2.40483
  j_{0,2}                           5.52008
  j_{0,3}                           8.65373
  k_1 = j01/(2sqrt2)                0.850234 m^-1
```

```
  k_2                              1.95164 m^-1
  k_3                              3.05955 m^-1
  omega_1 = c*k1                   2.54894e+08 rad/s
  omega_2 = c*k2                   5.85088e+08 rad/s
  omega_3 = c*k3                   9.17231e+08 rad/s
  T_1 = 2Pi/omega1                 2.46502e-08 s
  g_D (Dirac charge)               3.29106e-09 A*m
  J0(j01/sqrt2)                    0.397714
  A1 (field amplitude)             -2.91316e-23
  f(R ,0,0)                        0.397714
  f(2R,0,0) [BC check ~0]          8.80346e-10
  Asymptotic exponent J0 ~ r^p     p = -1/4
------------------------------------------------------------

All 10 sections and 12 plots completed successfully.
monopole_wave_maple.mpl —— TauTimp / Teoria Onduscularа
```